Tablet PC용 학습 및 필기 지원 플랫폼

이건희[0] 이경훈 류승하 전준호 정성택
한국공학대학교 컴퓨터공학과
{kunhee2926[0], sgn08062, tmdgk0524, juhajiju}@tukorea.ac.kr

The Platform for Learning and Writing Using Tablet PC

Geon-Hee Lee[0] Kyung-Hun Lee Seung-Ha Ryu Jun-Ho Jeon
Dept. of Computer Engineering, Tech University of Korea

요 약

본 논문에서는 단순 필기 기능에 머무르는 기존 필기 어플리케이션의 한계와 외부 프로그램에 대한 의존성으로 인한 불편함을 해결할 필요성을 기반으로, 다양한 기능을 통합한 필기 어플리케이션을 개발했습니다. 본 연구에서 개발한 어플리케이션은 필기 기능뿐만 아니라 요약, 번역, STT 기능을 지원하며, AI 모델을 탑재하여 온라인 환경에서 뿐만 아니라 오프라인 환경에서도 동작하도록 구현하였습니다.

또한, 이를 통해 사용자는 별도의 프로그램 전환 없이 하나의 디바이스에서 필기, 자료 정리, 학습 등 다양한 작업을 원활히 수행할 수 있으며, 학습 효율성과 편의성을 동시에 향상을 기대할 수 있습니다.

1. 서 론

최근 대학생들을 비롯한 많은 사람들이 학습 및 자료 정리를 목적으로 태블릿 PC를 활용하는 경우가 급격히 증가하고 있다.(인용) 태블릿 PC는 휴대성과 편리성을 바탕으로 기존의 노트북이나 종이 필기 도구를 대체하면서, 강의실과 도서관 등 다양한 학습 환경에서 필수적인 도구로 자리 잡고 있다.1)

그러나 현재 널리 사용되는 필기 어플리케이션들은 대부분 단순한 필기 기능에 국한되어 있으며, 요약이나 번역, 음성 텍스트 변환(STT)과 같은 학습에 유용한 기능을 제공하지 못하고 있다. 이로 인해 사용자는 필요한 기능을 활용하기 위해 외부 프로그램을 추가로 실행해야 하며, 이러한 과정은 학습 흐름을 방해하고 사용성을 저하시킨다.

우리는 이러한 필요성을 기반으로, 필기 기능과 함께 요약, 번역, STT 기능을 통합한 어플리케이션을 개발하였다. 특히, AI 모델을 적용하여 온라인 환경뿐만 아니라 오프라인 환경에서도 기능이 원활히 동작하도록 구현함으로써, 데이터 비용이나 네트워크 연결 상태에 대한 부담을 최소화하였고, 이를 통해 사용자의 외부 프로그램에 의존 문제를 해결하였다.

2. 관련 연구

2.1 기존 유사 앱과의 비교

	요약 기능	번역 기능	STT 기능	오프라인 지원	유료
플렉슬2)	X	▲	X	X	O
삼성 노트3)	X	X	O	O	X
노트 플러스	X	X	X	X	X
굿노트4)	X	X	O	X	O
옴니노트	O	O	O	O	X

표 1. 유사 어플리케이션과의 비교

위의 표 1은 기존에 사용되고 있는 대표적인 필기 어플리케이션과 우리가 개발한 앱(옴니노트)를 비교한 결과를 보여준다. 플렉슬의 경우 번역 기능을 일부 지원하는 것처럼 보이나, 이는 자체 기능이 아니라 외부 번역 어플을 연동해야만 사용할 수 있기 때문에 제한적이라는 의미에서 '△'로 표시하였다. 또한 오프라인 지원은 좌측의 요약 기능, 번역 기능, STT 기능이 네트워크 연결 없이도 정상적으로 작동하는지를 의미한다. 기존 앱들은 대부분 온라인 환경에 의존하고 있으며, 일부 기능만을 제한적으로 제공한다.

구체적으로 살펴보면, 삼성 노트와 굿노트는 STT 기능을 지원하지만 요약과 번역 기능이 없어서 학습 보조 도구로서의 활용 범위가 제한된다. 노트 플러스는 단순 필기 기능만을 제공하며 추가 기능이 전혀 제공되지 않는다.

그래서 옴니 노트는 외부 앱에 의존하지 않고 요약, 번역, STT 기능을 모두 자체적으로 제공하며, AI 모델을 적용하여 온라인뿐만 아니라 오프라인 환경에서도 동일하게 동작할 수 있는 통합형 앱을 개발하였다. 또한 무료로 제공되기 때문에 학습자들이 경제적 부담 없이 다양한 기능을 활용할 수 있다.

3. 세부 설계 및 구현

3.1 개발환경

본 논문의 어플리케이션은 윈도우 10에서 개발되었으며, Android Studio Meerkat을 기반으로 JDK 11, Android SDK, Gradle 8.9를 사

용하였다. 온라인 서버 환경은 Python을 기반으로 한 django 프레임워크를 사용하였다. 오프라인 로컬 서버 환경은 Termux를 기반으로 Flask 마이크로 웹 프레임워크를 사용하였다. 테스트에 사용한 단말기는 삼성전자의 갤럭시 탭 S8(Android 15)이고 해당 환경에서 테스트를 진행하였다.

3.2 시스템 구성

이 어플리케이션의 전체적인 동작 방식은 필기 어플(클라이언트)와 이를 지원하는 온라인/오프라인 서버 모듈이 상호작용하는 구조로 이루어진다. 사용자가 온라인 환경에서 앱에게 요청을 수행하면, 요청은 우선적으로 Web 서버(NGINX)에 전달되며, 정적 기능을 처리하거나 WSGI 서버(Gunicorn)으로 전달된다. 이후 WAS 서버(Django)에서 동적인 기능을 처리하고, 필요 시 MongoDB나 온라인 서버에 접근하여 수정된 파일을 백업하거나 불러오는 작업을 수행한다.

특히, 이 어플리케이션에서는 온라인 환경뿐 아니라 오프라인 환경에서도 기능이 동작하도록 설계하였다. 사용자가 네트워크 연결이 원활하지 않은 경우, 요청은 사용자 기기 내부의 로컬 서버(Flask)로 전달되며, 오프라인 AI 모델을 활용하여 번역, STT 기능을 수행할 수 있다. 이를 통해 데이터 연결 여부와 무관하게 핵심 기능을 안정적으로 제공한다. 전체적인 시스템 구성에서 온라인 시스템과 오프라인 시스템을 구분해서 아래 그림 1과 그림 2를 통해 한 눈에 확인할 수 있도록 정리하였다.

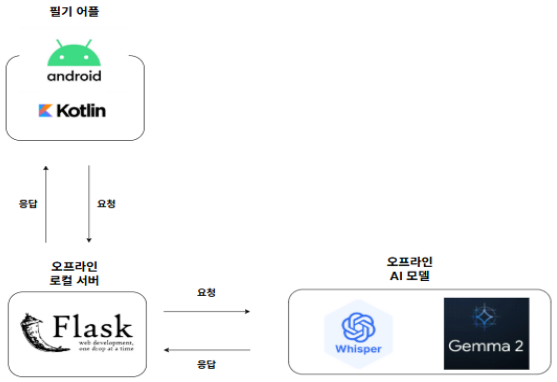

그림 1. 오프라인 시스템 구성도

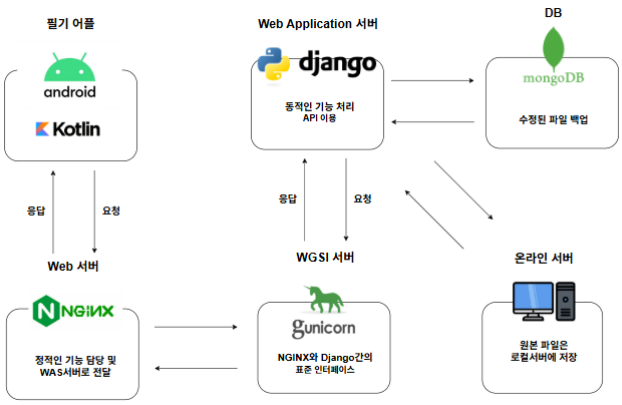

그림 2. 온라인 시스템 구성도

3.3 안드로이드에서의 구현

3.3.1 필기 데이터 생성, 제거 그리고 저장

```
when (e.action) {
    MotionEvent.ACTION_DOWN, MotionEvent.ACTION_MOVE -> {
        if(isEraserEnabled){
            performErase(e.x, e.y, xPdf, yPdf)
        }else{
            if(e.actionMasked == MotionEvent.ACTION_DOWN){
                currentStroke = Stroke(
                    color = currentColor,
                    width = currentWidth,
                    page = viewCurrentPage,
                    points = mutableListOf(PointF(xPdf, yPdf)),
                )
            }else{
                currentStroke?.points?.add(PointF(xPdf, yPdf))
            }
        }
    }
    MotionEvent.ACTION_UP    -> {
        if(!isEraserEnabled){
            currentStroke?.let { strokes.add(it)}
            currentStroke = null
        }
    }

DrawingView.kt
```

그림 3. DrawingView.kt의 일부분

PDF 기반 학습 환경에서 필기를 직접 PDF 위에 기록하는 경우, 원본 문서가 훼손되거나 수정이 어려운 문제가 발생한다. 이러한 한계를 해결하기 위해 본 어플리케이션은 PDF 원본과 필기 데이터를 분리하여 관리하는 방식을 채택하였다.

그림 3은 DrawingView.kt의 일부분으로, 사용자의 입력이 필기 데이터로 처리되는 과정을 보여준다. 본 시스템에서는 Stroke 객체를 통해 사용자가 그린 획을 관리하며, 각 Stroke는 색상, 굵기, 페이지 번호, 좌표 리스트 등의 정보를 포함한다. 이를 통해 원본 PDF는 변경하지 않고, 필기 정보만 별도의 계층에 저장할 수 있다.

따라서 사용자가 화면에 펜 입력을 수행하면, 애플리케이션은 먼저 터치 좌표를 PDF 좌표계로 변환한 뒤, 새로운 Stroke 객체를 생성하여 해당 페이지의 필기 목록에 추가한다. 이후 onDraw 메서드를 통해 Stroke들이 순차적으로 렌더링되며, 사용자는 마치 원본 PDF 위에 직접 필기하는 것처럼 시각적으로 확인할 수 있다.

또한, 지우개 기능이 활성화되면 특정 반경 내의 Stroke 세그먼트를 삭제하고, 남은 세그먼트를 새로운 Stroke로 재구성함으로써 세밀한 수정이 가능하다. 최종적으로 모든 필기 데이터는 JSON 형식[5]으로 직렬화되어 PDF와 함께 .mydoc 포맷으로 압축·저장되며, 복원 시에는 원본 PDF와 분리된 필기 데이터를 다시 불러와 정확히 재현할 수 있다.

3.4 온라인 서버에서의 구현

3.4.1 사용자 인증

사용자 인증 기능은 Django의 기본 User 모델을 기반으로 하며, SimpleJWT[6]를 이용해 토큰으로 사용자 인증하는 방식을 채택하였다. 회원가입, 로그인, 로그아웃, 토큰 재발급을 위한 API가 구현되어 있으며, 모든 서비스 접근에는 JWT 기반 인증이 필수적이다.아래 그림 2는 그 과정을 나타낸 소스의 일부분이다

```
# apps/users/views.py
class SignupView(APIView):
    def post(self, request):
        serializer = UserSignupSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response({'message': 'User registered successfully'}, status=201)
        return Response(serializer.errors, status=400)
class LoginView(APIView):
    def post(self, request):
        serializer = UserLoginSerializer(data=request.data)
        if serializer.is_valid():
            user = authenticate(...)
            refresh = RefreshToken.for_user(user)
            return Response({'access': str(refresh.access_token), 'refresh':
str(refresh)})
        return Response(serializer.errors, status=400)
```

그림 4. User 모델 기반 JWT 인증 코드 중 일부

3.4.2 파일 및 문서 관리

파일 관리 기능은 사용자가 업로드한 파일을 {user_id}/{file_type}/… 구조의 디렉터리에 저장하고, Django 모델을 통해 메타데이터를 관리한다. 이를 통해 문서, 이미지, 음성 파일 등 다양한 학습 자산을 효율적으로 분류하고 관리할 수 있다. 또한, 업로드·목록 조회·다운로드·삭제와 같은 기본 기능을 모두 제공한다.

문서 관리 기능은 STT, 요약 등의 결과 파일을 비롯한 텍스트 자산을 통합적으로 관리한다. 불필요해진 문서는 API를 통해 삭제할 수 있으며, 서버에서는 실제 파일과 DB 레코드를 동시에 제거하여 일관성을 보장한다.

3.4.3 핵심 기능(STT, 요약 기능)

```
# apps/stt/urls.py
urlpatterns = [
    path('', STTProcessView.as_view(), name='stt-process'),
    path('result/<str:task_id>/', STTResultView.as_view(), name='stt-result'),
]
# apps/stt/views.py
class STTProcessView(APIView):
    permission_classes = [IsAuthenticated]
    def post(self, request):
        audio_file = request.FILES.get('audio_file')
        # 확장자 검증 등...
        # 저장 후 Celery 태스크 실행
        task = transcribe_audio.delay(saved_path)
        return Response({"task_id": task.id}, status=202)

class STTResultView(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request, task_id):
        task_result = AsyncResult(task_id)
        # processing / completed / failed 상태 응답
```

그림 5. Whisper 모델 기반 STT 기능 구현 코드 중 일부

STT(Speech-to-Text) 기능은 Whisper 모델[7]을 기반으로 구현되었다. 사용자가 음성 파일을 업로드하면, Celery 태스크 큐에 작업이 등록되고, 워커가 Whisper 모델을 호출하여 한국어 음성을 텍스트로 변환한다. 결과는 비동기적으로 반환되며, 사용자는 task_id를 통해 처리 상태(processing/completed/failed) 및 최종 결과를 조회할 수 있다.

```
# apps/summarization/urls.py
urlpatterns = [
    path("", SummarizeTextView.as_view(), name="summarize_text"),
    path("textfile/", SummarizeTextFileView.as_view(), name="summarize_textfile"),
    path("pdf/", SummarizePDFView.as_view(), name="summarize_pdf"),
    path("result/<str:task_id>/", SummarizeResultView.as_view(),
name="summarize_result"),
]

# apps/summarization/tasks.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
@lru_cache(maxsize=1)
def load_model():
    tokenizer = AutoTokenizer.from_pretrained("wisenut-nlp-team/KoT5",
use_auth_token=HUGGINGFACE_TOKEN)
    model = AutoModelForSeq2SeqLM.from_pretrained("wisenut-nlp-team/KoT5",
use_auth_token=HUGGINGFACE_TOKEN).to(DEVICE)
    return tokenizer, model

@shared_task
def summarize_text_kot5(text:str) -> str:
    tokenizer, model = load_model()
    inputs = tokenizer([text], max_length=1024, truncation=True,
return_tensors="pt").to(DEVICE)
    summary_ids = model.generate(**inputs, max_length=150, min_length=30,
num_beams=2, length_penalty=2.0, early_stopping=True)
    return tokenizer.decode(summary_ids[0], skip_special_tokens=True)
```

그림 6. KoT5 모델 기반 요약 기능 구현 코드 중 일부

요약(Summarization) 기능은 KoT5 모델[8]을 활용한다. 입력 텍스트뿐만 아니라 텍스트 파일 및 PDF에서 추출된 텍스트도 요약할 수 있으며, 역시 Celery 기반 비동기 처리 방식을 따른다. 작업 요청 후 반환되는 task_id를 통해 결과를 조회할 수 있으며, KoT5 모델은 GPU 사용 환경에서는 CUDA, 그렇지 않을 경우 CPU에서 동작한다.

3.4.4 비동기/분산 인프라 구조

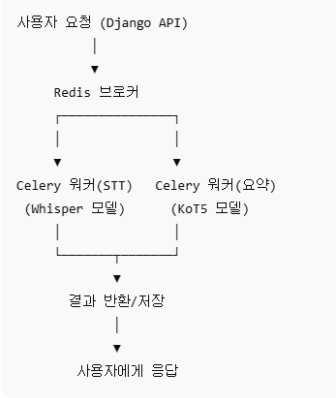

그림 7. 서버 비동기/분산 인프라 구조 순서도 표현

서버 전체는 비동기/분산 인프라 구조로 동작한다. Django가 요청을 수신하면 Redis 브로커에 태스크를 등록하고, Celery 워커가 STT 또는 요약 모델을 실행한 뒤 결과를 반환한다. 이를 통해 대규모 요청에도 안정적인 처리가 가능하다. 이후 결과를 반환하거나 저장하여 사용자의 요청에 대한 응답을 보낸다.

3.5 오프라인 서버에서의 구현

3.5.1 번역 기능

```
# ====== [1. 텍스트 번역 기능] ======
@app.route('/translate', methods=['POST'])
def translate():
    try:
        data = request.get_json()
        user_input = data.get("text", "").strip()

        if not user_input:
            return jsonify({"error": "입력 텍스트가 비어 있습니다."}), 400

        # llama.cpp 프롬프트 구성
        prompt = f'Translate this from English to Korean:\\nEnglish: {user_input}\\nKorean:'
        command = f'./llama.cpp/build/bin/llama-cli -m ~/gemmax2-28-2b-v0.1-q4_k_m.gguf -p "{prompt}" -n 512 -no-cnv > {TRANSLATE_OUTPUT}'

        # 번역이 실행
        subprocess.run(command, shell=True, check=True)
        time.sleep(1)

        # 결과 읽기
        with open(TRANSLATE_OUTPUT, "r", encoding="utf-8") as f:
            result = f.read()

        return jsonify({"result": result.strip()})

    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

그림 8. 오프라인 로컬 서버 번역 기능 구현 코드 중 일부

/translate 엔드포인트는 사용자가 입력한 원문 텍스트를 받아, 지정된 번역 모델을 호출하여 목적 언어(한국어)로 변환한다. 이 과정은 사전 학습된 다국어 번역 모델을 이용하며, 요청 본문은 JSON 포맷 {“text”: … }으로 구성된다. 응답 또한 JSON 형태로 반환되어, 안드로이드 클라이언트가 쉽게 파싱할 수 있도록 하였다. 번역 연산은 상대적으로 짧은 시간에 처리가 가능하도록 기존 모델에서 정확도를 약간 포기하는 대신 처리 속도를 향상시켰다.

3.5.2 STT 기능

```python
# ====== [2. STT 음성 인식 기능] ======
@app.route('/upload', methods=['POST'])
def upload_file():
    try:
        file = request.files.get('file')
        if not file:
            print(" [Flask] 파일이 request에 없음.")
            return jsonify({'error': '파일이 없습니다.'}), 400

        filename = file.filename
        if not filename:
            print(" [Flask] 파일명 없음.")
            return jsonify({'error': '파일 이름이 비어 있습니다.'}), 400

        print(f" [Flask] 파일 저장 경로: {save_path}")


        # Whisper 실행
        command = f"./whisper.cpp/build/bin/whisper-cli -m {WHISPER_MODEL_PATH} -l ko -f {save_path} > {result_path}"
        print(f" [Flask] 명령 실행: {command}")

        if result.returncode != 0:
            return jsonify({'error': 'Whisper 실행 실패', 'stderr': result.stderr}), 500

        # 결과 파일 읽기
        if not os.path.exists(result_path):
            print(" 결과 파일 없음")
            return jsonify({'error': 'STT 결과 파일이 존재하지 않습니다.'}), 500

        with open(result_path, "r", encoding="utf-8") as f:
            result_text = f.read()

        print(" 결과 파일 읽기 완료")
        return jsonify({'stt_result': result_text.strip()})

    except Exception as e:
        print(f" 예외 발생: {e}")
        return jsonify({'error': str(e)}), 500
```

그림 9. 오프라인 로컬 서버 STT 기능 구현 코드 중 일부

서버는 /upload 엔드포인트를 통해 클라이언트로부터 음성 파일을 수신한다. 수신된 파일은 서버의 임시 저장소에 저장된 후, 음성 인식 엔진을 호출하여 텍스트로 변환된다. STT 연산은 비교적 장시간이 소요될 수 있기 때문에, 서버는 요청마다 고유한 task_id를 부여하고, 클라이언트는 이를 기반으로 후속 조회를 통해 결과를 확인할 수 있도록 설계하였다.

3.5.3 안정성 및 예외 처리

오프라인 서버는 모든 엔드포인트에서 오류 발생 시 예외를 잡아 JSON 형태의 에러 메세지를 반환한다. 파일 형식 오류, 모델 실행 중단, 통신 연결 문제 등 다양한 상황을 고려하여 에러 핸들링을 구성하였다. 또한, 요청-응답 구조를 단순화하여 안드로이드 클라이언트가 쉽게 연동할 수 있도록 하였으며, 장시간 연산이 필요한 경우에는 비동기 처리를 통해 서버의 안정성을 유지하였다.

3.6 구현결과

온라인/오프라인 서버와 클라이언트 간의 통신이나 기본적인 필기 기능은 모두 사용이 가능할 정도로 구현하였다. 그리고 번역, 요약, STT 등과 같은 핵심적인 기능도 마찬가지로 사용이 가능할 정도로 구현하였다. 하지만 구현 과정에서 하나의 어플리케이션에서 AI 모델을 직접 실행하려 했으나, AI 모델을 실행하기 위한 환경 구축을 위해 필수적인 C++ 라이브러리[9]를 안드로이드에서 지원하지 않아 Termux 환경을 활용하여 오프라인에서도 AI 모델이 동작하도록 하였으나, 하나의 앱 내에서 모든 기능을 처리하지 못한 점은 아쉬움으로 남는다.

결국, 앞서 이야기 하였던 외부 의존성 문제는 남았지만, 유사 앱 간의 기능에 대한 부분에 차별성을 가지는 필기 앱을 만드는데 성공하였다. 아래 그림 10에 구현된 결과와 어플리케이션 테스트 화면이 있다.

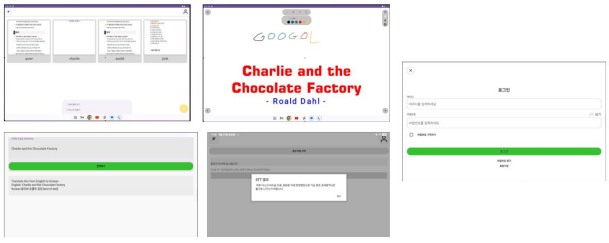

그림 10. 구현 결과 및 테스트

4. 결론 및 향후 연구과제

본 앱은 기존 필기 앱이 단순 필기 기능에 머무르는 한계를 극복하기 위해 필기 기능과 함께 요약·번역·STT 기능을 통합한 안드로이드 기반 학습 보조 어플리케이션을 개발하였다. 개발된 어플리케이션은 특히 AI 모델을 활용한 학습 기능을 온라인·오프라인 환경 모두에서 지원함으로써 학습 효율성과 편의성을 높였다.

다만, 안드로이드 환경에서 Android NDK를 통해 C++ 네이티브 코드를 실행할 수는 있으나, AI 모델을 하나의 애플리케이션 내부에서 구동하기에는 메모리 자원과 실행 파일 관리 측면에서 한계가 있었다. 특히 Android 10(API level 29) 이상에서는 보안 정책[10]으로 인해 앱 디렉토리 내 실행 파일 실행이 제한된다는 점이 공식 문서[11]를 통해 확인되었다. 이러한 제약으로 인해 최종적으로는 안드로이드에서 리눅스 환경을 제공하는 Termux를 활용하여 AI 모델을 구동하는 방식을 채택하였다. 이를 통해 단말기 내에서 대규모 모델을 오프라인으로 실행하면서도 애플리케이션과의 연동을 가능하게 하였으나, 이는 여전히 외부 의존성을 가진다는 한계가 있다.

향후 연구에서는 Termux 의존도를 줄이고, Android NDK 및 GPU 가속을 활용하여 앱 내부에서 AI 모델을 경량화·최적화하여 실행할 수 있는 방법을 모색할 필요가 있다. 또한, 모델 다운로드 및 관리 방식을 간편화하여 사용자가 보다 안정적이고 효율적으로 오프라인 AI 서비스를 이용할 수 있도록 개선하는 것이 필요하다. 더 나아가 다양한 플랫폼 환경에서의 이식성 확보와, 모바일 디바이스의 연산 자원 한계를 극복하기 위한 방법에 대한 연구도 필요해 보인다.

참고문헌

1) 조아라, 김세린 기자, 「노트북, 스마트폰에치이더니 '수요 폭발'…대학생'필수품'됐다」 ,한국경제신문, 2022.12.14, https://www.hankyung.com/article/202212145709g
2) Flexcil, Flexcil Help Center, "처음 사용자를 위한 가이드", https://support.flexcil.com/hc/ko/articles/6696626355609-처음-사용자를-위한-가이드, 2023. 08. 26
3) 삼성전자, 삼성전자서비스, "삼성 노트 사용 방법", https://www.samsungsvc.co.kr/solution/38425, 2022. 03. 02
4) 굿노트, Goodnotes, "새로운 노트 작성 방법을 소개합니다.", https://www.goodnotes.com/kr/features
5) Json, EMCA-404, "JSON 개요", json.org/josn-ko.html
6) David Sanders, django-rest-framework, "Simple JWT", https://django-rest-framework-simplejwt.readthedocs.io/en/latest/#
7) Jongwook, Github, "Whisper", https://github.com/openai/whisper
8) Psyche, Huggingface.co, "KoT5-summarization", https://huggingface.co/psyche/KoT5-summarization
9) Google, Developers, "C++ 라이브러리 지원", https://developer.android.com/ndk/guides/cpp-support?utm_source=chatgpt.com&hl=ko
10) Google, Developers, "동작 변경사항: API 레벨 29 이상을 타겟팅하는 앱", https://developer.android.com/about/versions/10/behavior-changes-10?utm_source=chatgpt.com&hl=ko#foldables
11) Google, Developers, "앱 보안 강화", https://developer.android.com/privacy-and-security/security-best-practices?utm_source=chatgpt.com&hl=ko